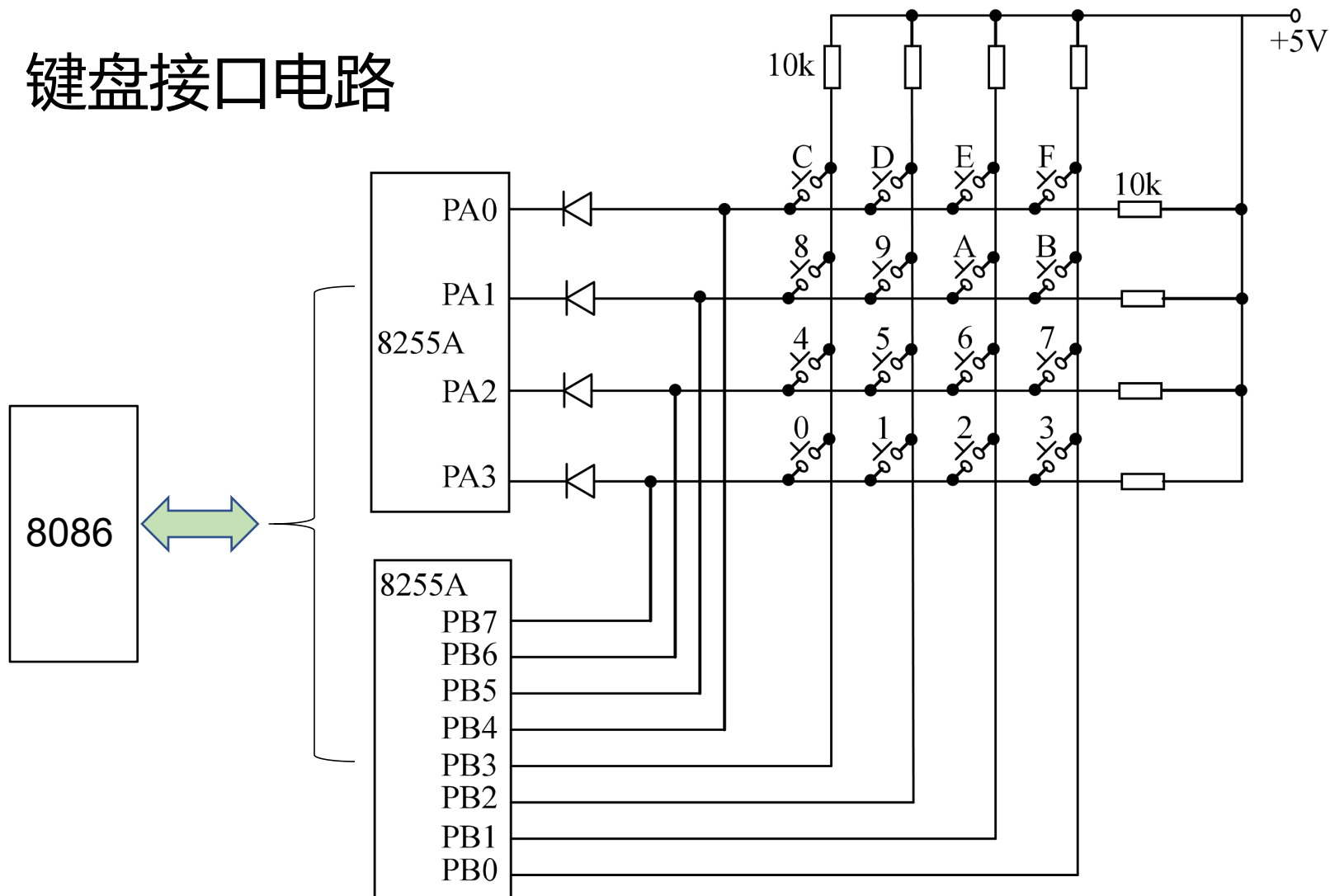




6.3.3 键盘接口

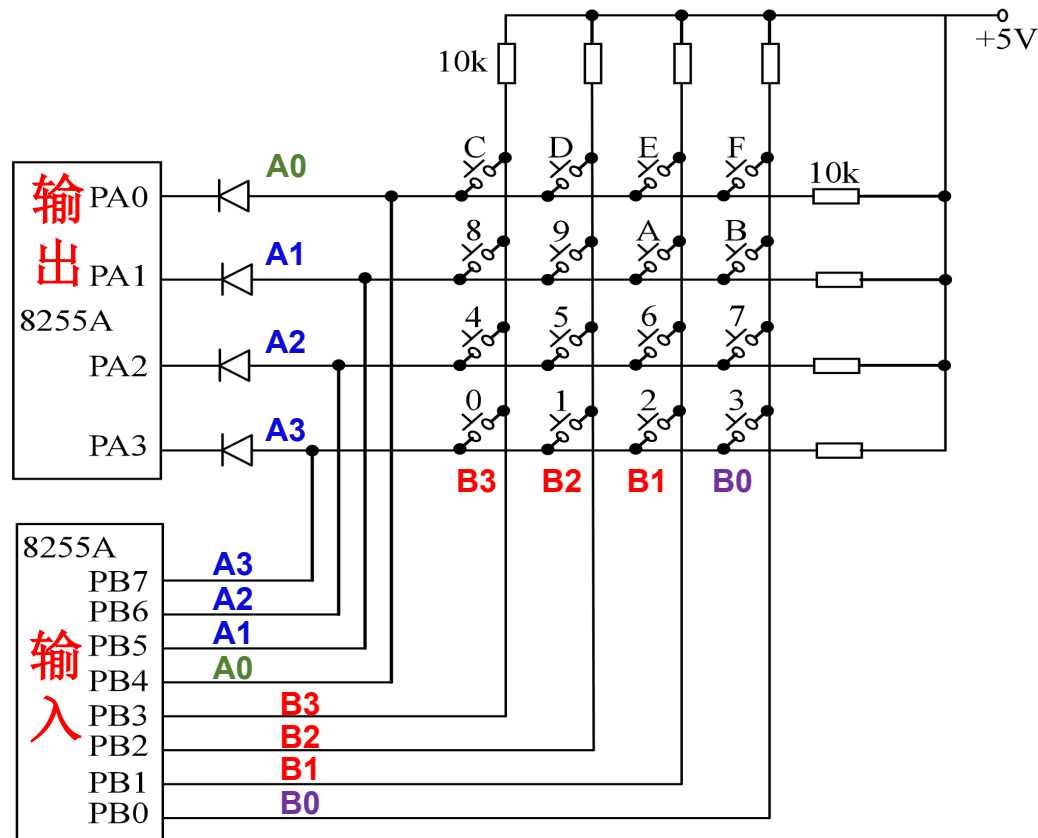
键盘接口电路



4*4键盘工作原理

- ① PA/PB工作在方式0;
PA输出, PB输入;

8086



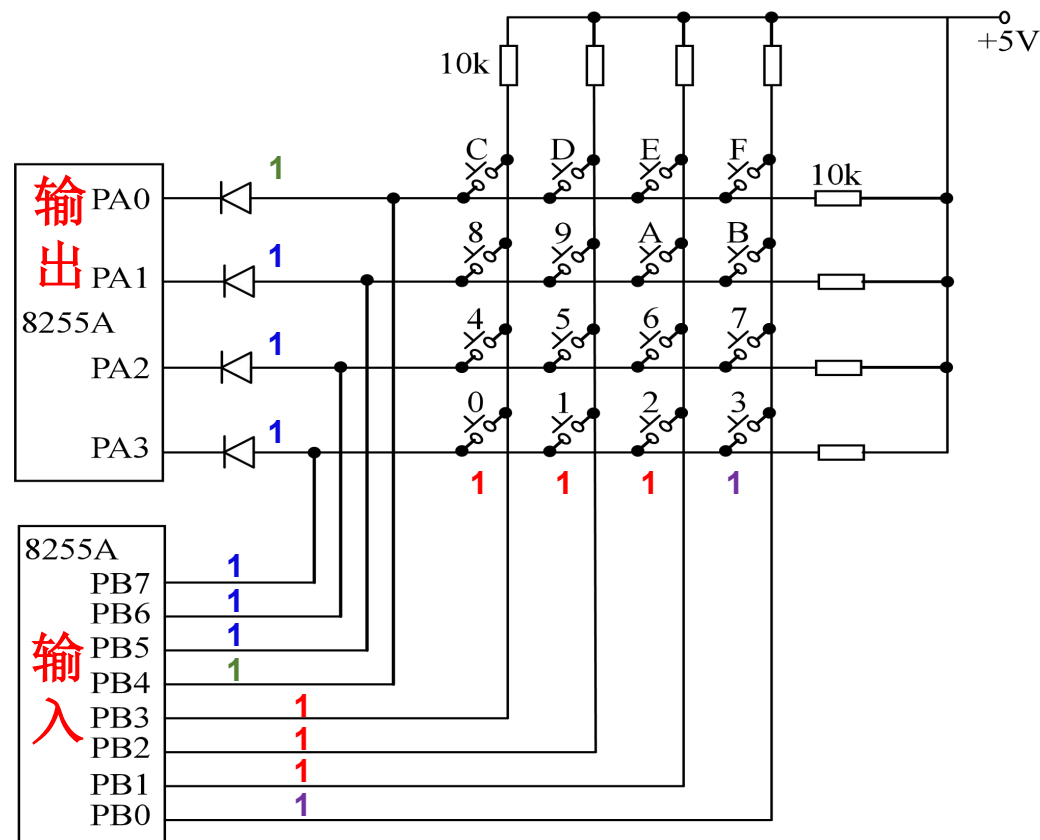
- ② 从PB读入的数据特点:

- PB7-4 = PA3-0, 代表4*4键盘的行电平状态
- PB3-0, 代表4*4键盘的列电压状态, 其值与PA3-0的输出、按键是否按下有关。

4*4键盘工作原理

- PA3-0输出1111B时，则无论有无按键按下，PB3-0读入都为1111B，无法据此判断是否有键按下。

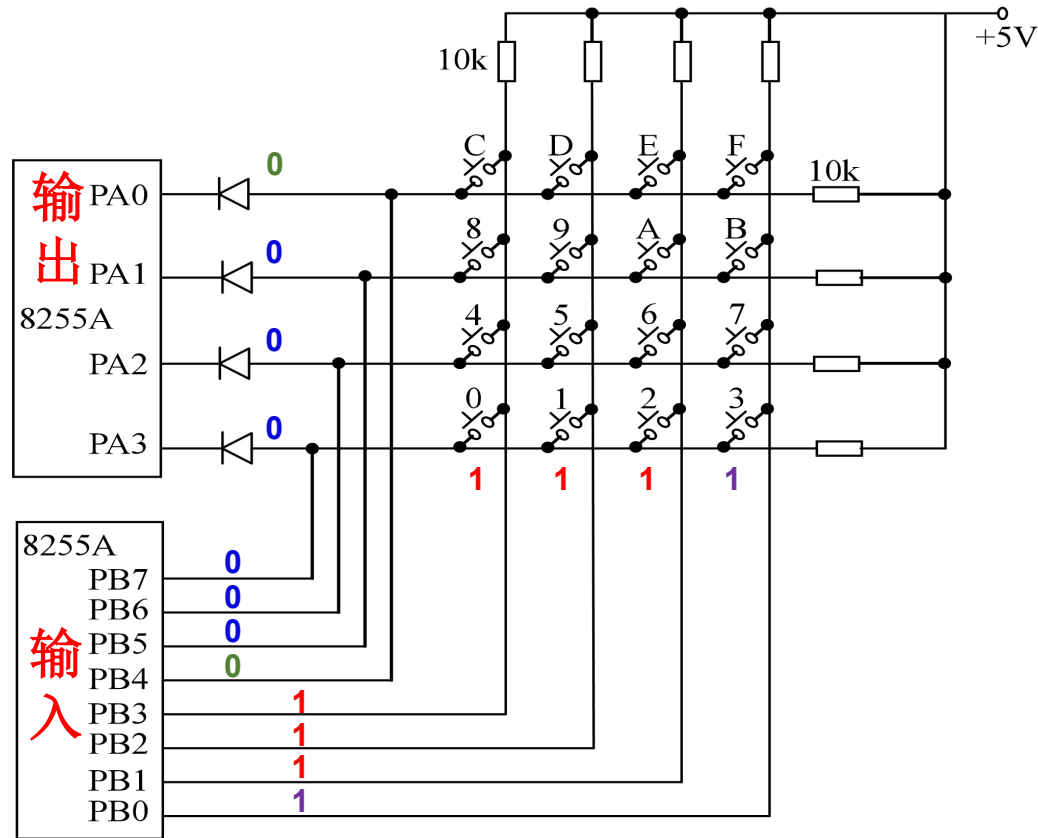
8086



4*4键盘工作原理

- PA3-0输出1111B时，则无论有无按键按下，PB3-0读入都为1111B，无法据此判断是否有键按下。

8086



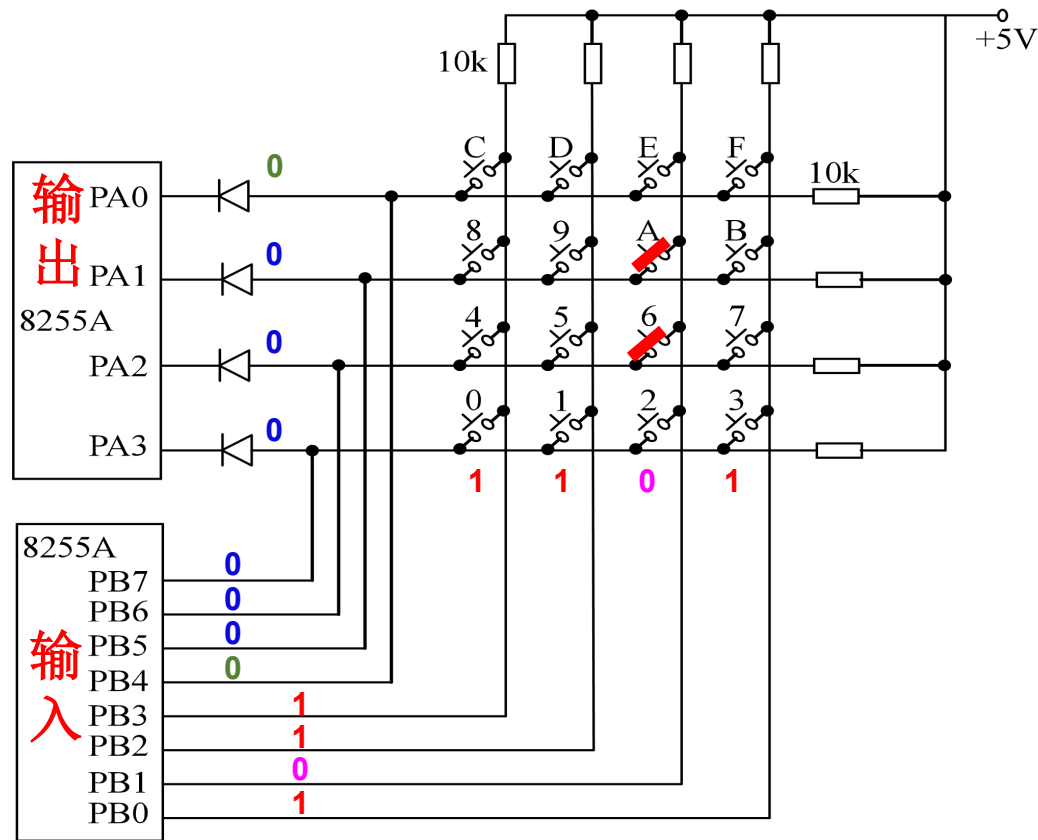
- 判断哪个按键按下的原理：

① PA3-0输出0000B，如果PB3-0读入为1111B，则表明无键按下

4*4键盘工作原理

- PA3-0输出1111B时，则无论有无按键按下，PB3-0读入都为1111B，无法据此判断是否有键按下。

8086



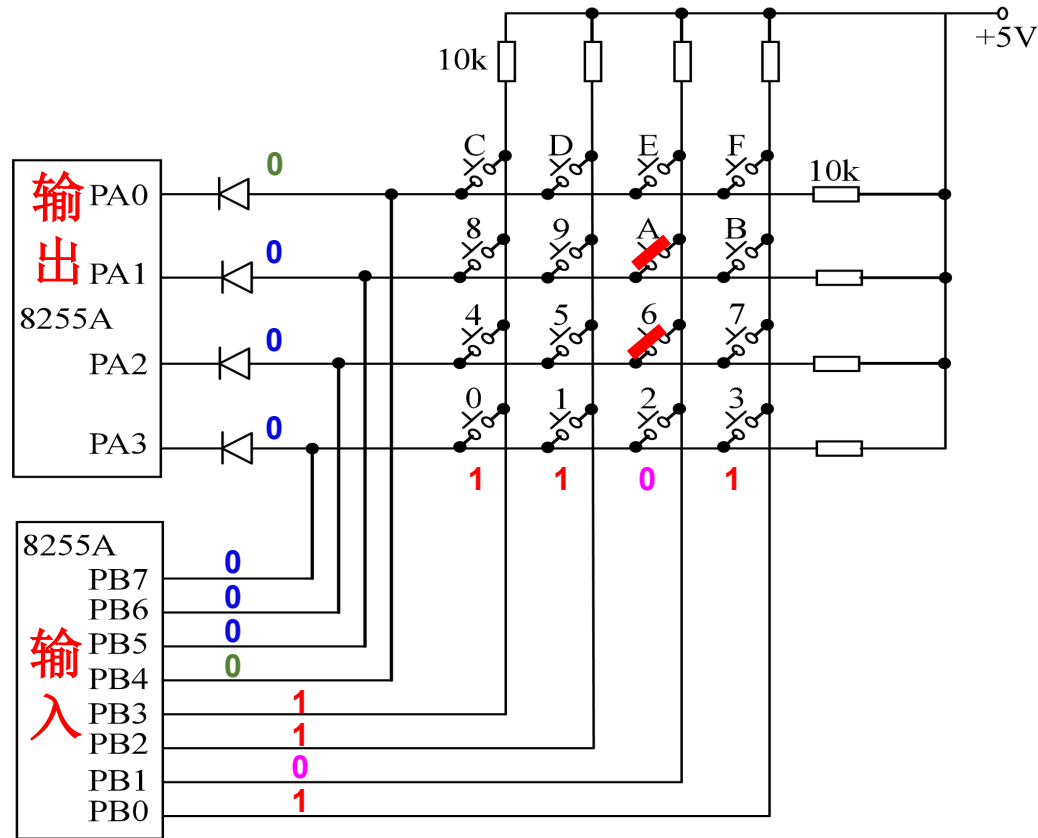
- 判断哪个按键按下的原理：

- ① PA3-0输出0000B，如果PB3-0读入为1111B，则表明无键按下
- ② PA3-0输出0000B，如果PB3-0读入值不等于1111B，则表明有键按下（但此时无法判断哪个键按下）

4*4键盘工作原理

- PA3-0输出1111B时，则无论有无按键按下，PB3-0读入都为1111B，无法据此判断是否有键按下。

8086



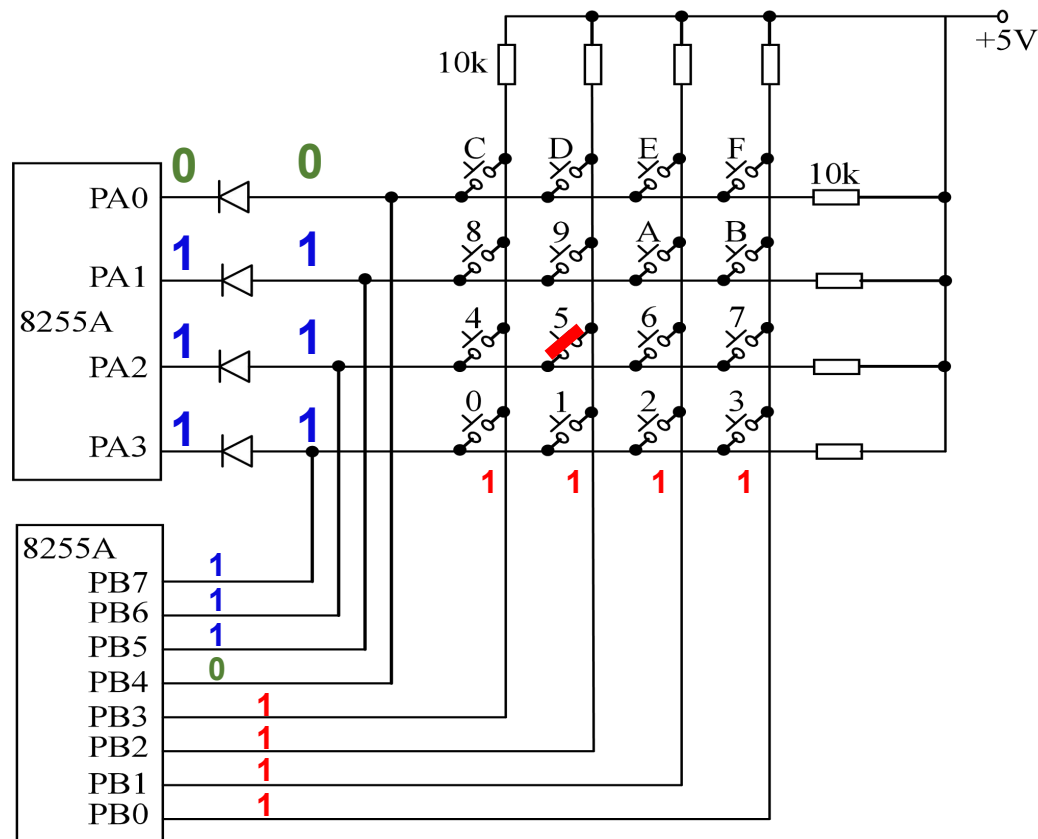
- 判断哪个按键按下的原理：

- ① PA3-0输出0000B，如果PB3-0读入为1111B，则表明无键按下
- ② PA3-0输出0000B，如果PB3-0读入值不等于1111B，则表明有键按下（但此时无法判断哪个键按下）
- ③ 确定哪个键按下：使得4*4键盘的每个行的电压依次为0（即PA3、2、1、0四个引脚依次输出0，同时其它引脚为1），读取PB。如果该行有键按下，则对应的列电平也会被拉低为0（PB3-0中对应的引脚为0）-- 同时获取按键**扫描码**。

扫描键盘、确定扫描码

假设 **5键** 按下。则向PA0~PA3
依次输出0，然后读PB端口。

8086 

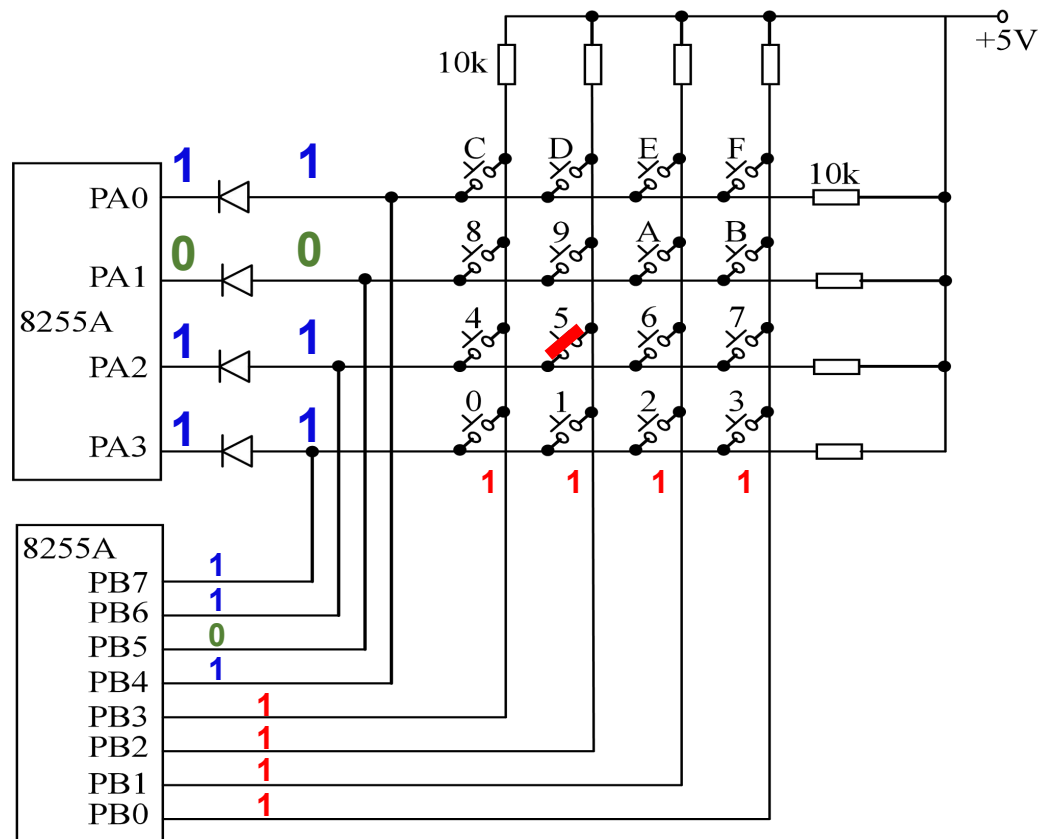


① PA3~PA0 = **1110**B, 从PB读入的数据为**1110 1111**B。低4位全1表明按键不在第0行

扫描键盘、确定扫描码

假设 **5键** 按下。则向PA0~PA3
依次输出0，然后读PB端口。

8086

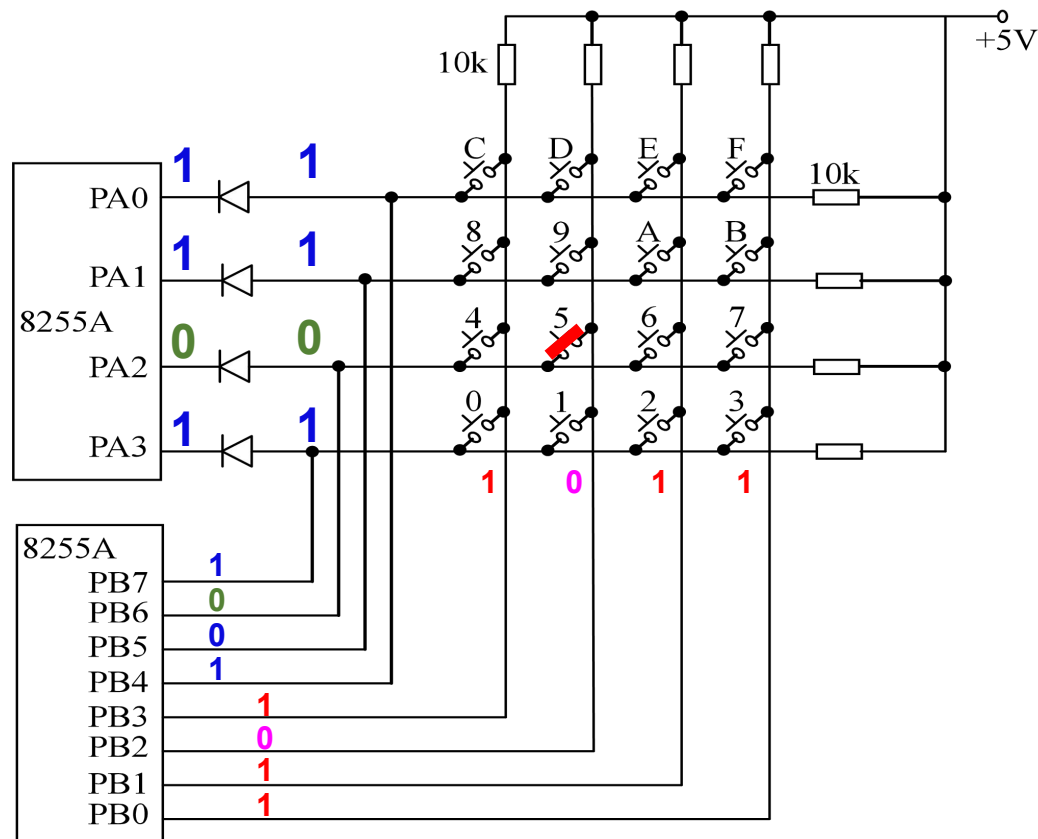


- ① PA3~PA0 = **1110**B, 从PB读入的数据为**1110** **1111**B。低4位全1表明按键不在第0行
- ② PA3~PA0 = **1101**B, 从PB读入的数据为**1101** **1111**B。低4位全1表明按键不在第1行

扫描键盘、确定扫描码

假设 **5键** 按下。则向PA0~PA3
依次输出0，然后读PB端口。

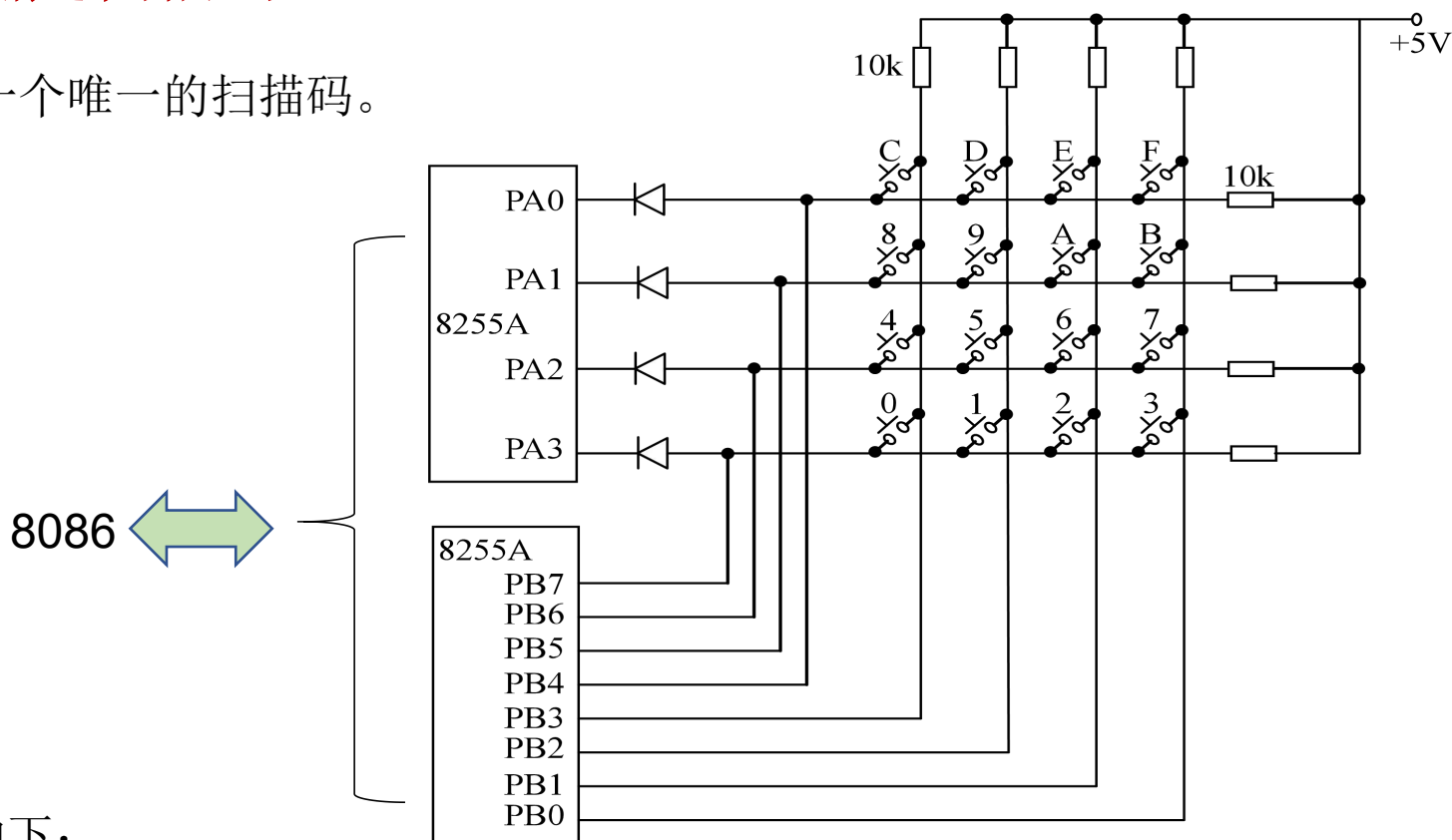
8086



- ① PA3~PA0 = **1110**B, 从PB读入的数据为**1110 1111**B。低4位全1表明按键不在第0行
- ② PA3~PA0 = **1101**B, 从PB读入的数据为**1101 1111**B。低4位全1表明按键不在第1行
- ③ PA3~PA0 = **1011**B, 从PB读入的数据为**1011 1011**B。低4位不全1表明按键在第2行。
并且**1011 1011**B (BBH) 可确定按下的键为5。BBH即为按键5的扫描码

扫描键盘、确定扫描码

■ 每个按键对应一个唯一的扫描码。



■ 0~F的扫描码如下：

```
;          0      1      2      3      4      5      6      7
TABLE      DB  77H,7BH,7DH,7EH,0B7H,0BBH,0BDH,0BEH
;          8      9      A      B      C      D      E      F
          DB  0D7H,0DBH,0DDH,0DEH,0E7H,0EBH,0EDH,0EEH
```

程序编写步骤（主要部分）：

1、端口定义（设端口为0FF9H~0FFFH）

2、数据段定义

3、8255A初始化

4、准备工作

1) 检测是否所有键都松开，若没有则反复检测

2) 当所有键都松开时，再检测是否有键压下，若没有则反复检测

3) 若有键按下，延迟20ms再检查是否依然按下，以消除干扰。

如果没有键按下，则返回2)

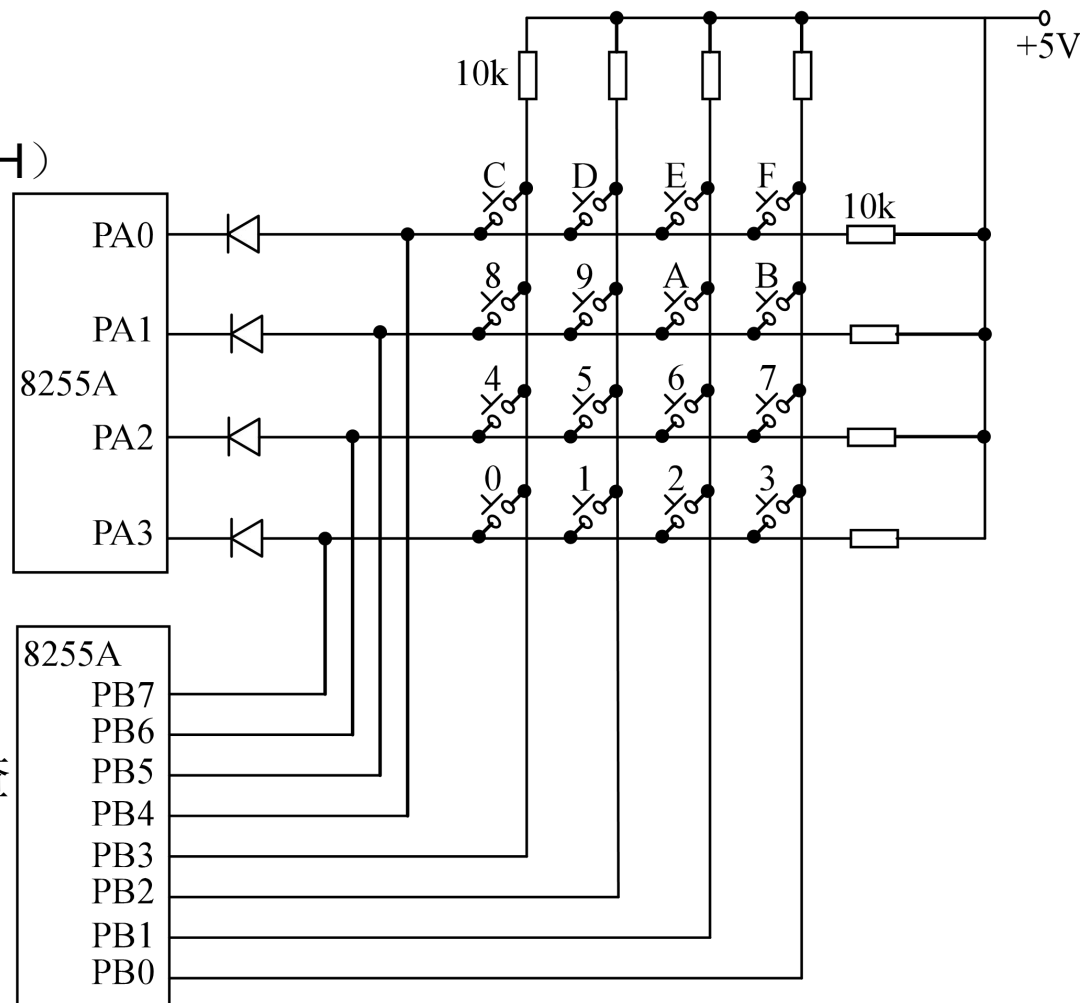
如果依然有键按下，则进行步骤5

5、扫描定位

PA0~PA3依次输出0，检查PB低4位，定位按键所在的行

6、获取扫描码，并转换为键值

从PB获取扫描码，与数据段中预定义的扫描码对比，确定按键值



;1.定义端口地址

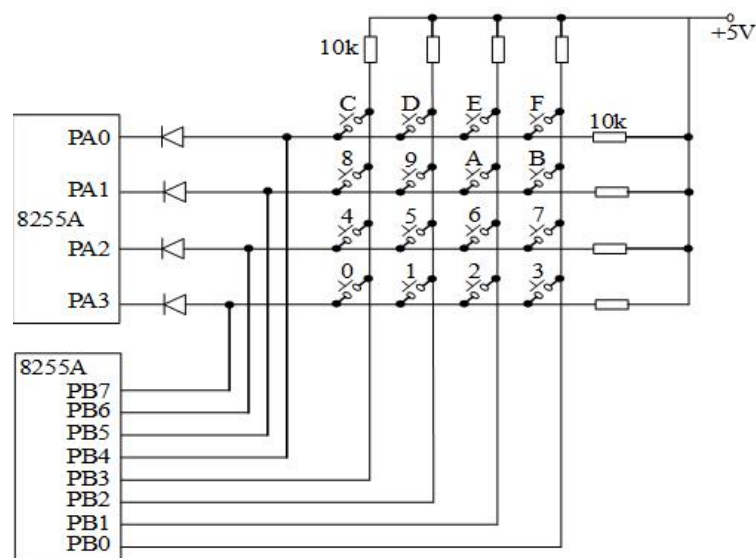
```
PORT_A      EQU    0FF9H      ;8255A端口A地址
PORT_B      EQU    0FFBH      ;8255A端口B地址
PORT_CTL    EQU    0FFFH      ;8255A控制口地址
```

;2.定义数据段, 键盘扫描码表

```
DATA        SEGMENT
;           0    1    2    3    4    5    6    7
TABLE       DB  77H,7BH,7DH,7EH,0B7H,0BBH,0BDH,0BEH
;           8    9    A    B    C    D    E    F
           DB  0D7H,0DBH,0DDH,0DEH,0E7H,0EBH,0EDH,0EEH
DATA        ENDS
```

;2.1 定义堆栈段

```
STK1        SEGMENT STACK
SDT         DW  50 DUP(0)
TOP_STK     EQU SIZE SDT
STK1        ENDS
```



;3 定义代码段

CODE SEGMENT

ASSUME CS:CODE, DS:DATA, SS:STK1

```
START: MOV     AX, STACK
        MOV     SS, AX
        MOV     SP, TOP_STK
        MOV     AX, DATA
        MOV     DS, AX
```

;初始化8255A, 方式0, A口作输出, 端口B和C作输入

```
        MOV     DX, PORT_CTL    ;指向控制口
        MOV     AL, 1000 1011B  ;控制字8BH
        OUT     DX, AL          ;写入控制字
```

;向所有行送0

```
        MOV     DX, PORT_A      ;
```

```
        MOV     AL, 00H
```

```
        OUT     DX, AL          ;向端口A各位输出0
```

;读列, 察看是否所有键松开

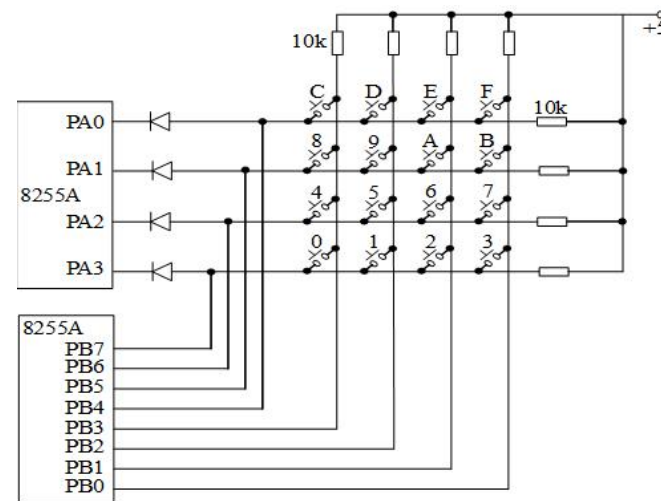
```
        MOV     DX, PORT_B
```

```
WAIT_OPEN: IN  AL, DX           ;从B口读取键盘状态
```

```
        AND     AL, 0FH         ;只查低4位
```

```
        CMP     AL, 0FH        ;是否都为1(键都松开)
```

```
        JNE     WAIT_OPEN      ;否继续查
```



;各键都松开，再查列是否有0，即：是否有键按下

```
WAIT_PRESS: IN      AL, DX                ;读B口
              AND    AL, 0FH              ;
              CMP    AL, 0FH              ;
              JE     WAIT_PRESS           ;无，等待
```

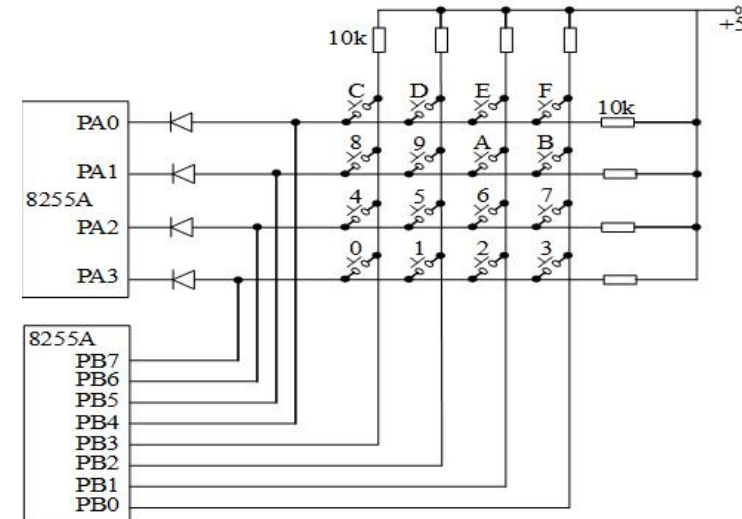
;有键按下，延时20ms，消除抖动

```
MOV    CX, 16EAH
```

```
DELAY:    LOOP DELAY    ;
```

;再查列，看键是否还按下

```
IN      AL, DX
AND     AL, 0FH
CMP     AL, 0FH
JE      WAIT_PRESS ;已松开，转去等待压键
```



;键仍然被压着，扫描PA0~1，确定是哪个键按下

MOV AL, 0FEH ;先使最低位D0=0

MOV CL, AL ;CL=1111 1110B

NEXT_ROW:

MOV DX, PORT_A ;

OUT DX, AL ;向某一行输出低电平

MOV DX, PORT_B ;

IN AL, DX ;读取B口状态

AND AL, 0FH ;只截取列值

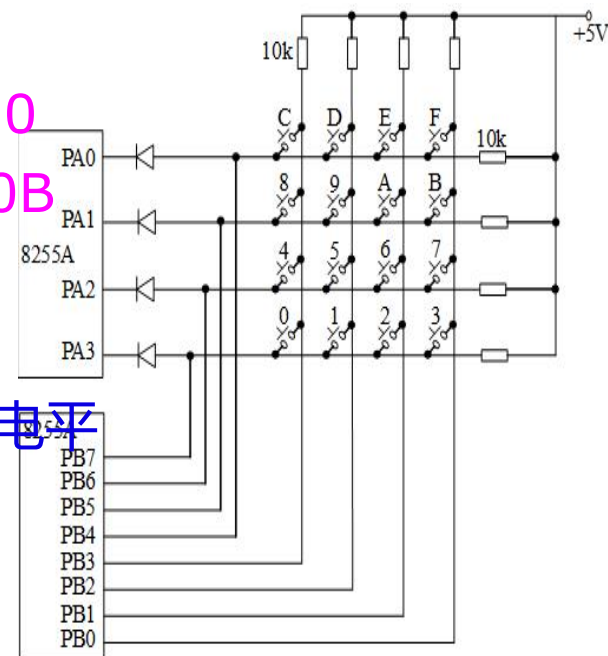
CMP AL, 0FH ;是否都是1

JNE ENCODE ;否，表示有键按下，转去编码

ROL CL, 01 ;是，则循环移位，使下一行输出0

MOV AL, CL

JMP NEX_ROW ;扫描下一行



			0	1	2	3	4	5	6	7
;	TABLE	DB	77H,7BH,7DH,7EH,0B7H,0BBH,0BDH,0BEH							
;			8	9	A	B	C	D	E	F
		DB	0D7H,0DBH,0DDH,0DEH,0E7H,0EBH,0EDH,0EEH							

;已找到有一列为低电平，对压键的行列值编码

ENCODE:

```
IN      AL, DX      ;从B口读取行列值
MOV     BX, 000FH   ;初值F,使得TABLE[BX]指向F的扫描码
```

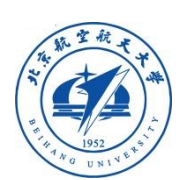
NEXT_TRY:

```
CMP     AL, TABLE[BX] ;读入的行列值与表中的扫描码比较
JE      DONE           ;相等，转出
DEC     BX             ;不等，指向前一个数的扫描码
JNS     NEXT_TRY       ;若地址尚未减为负值，则继续
MOV     AH, 01         ;若为负值，AH置出错码01
JMP     EXIT           ;退出
```

```
DONE: MOV     AL, BL      ;BL中为按键对应的16进制数值
      MOV     AH, 00     ;AH=0,读到有效键值
```

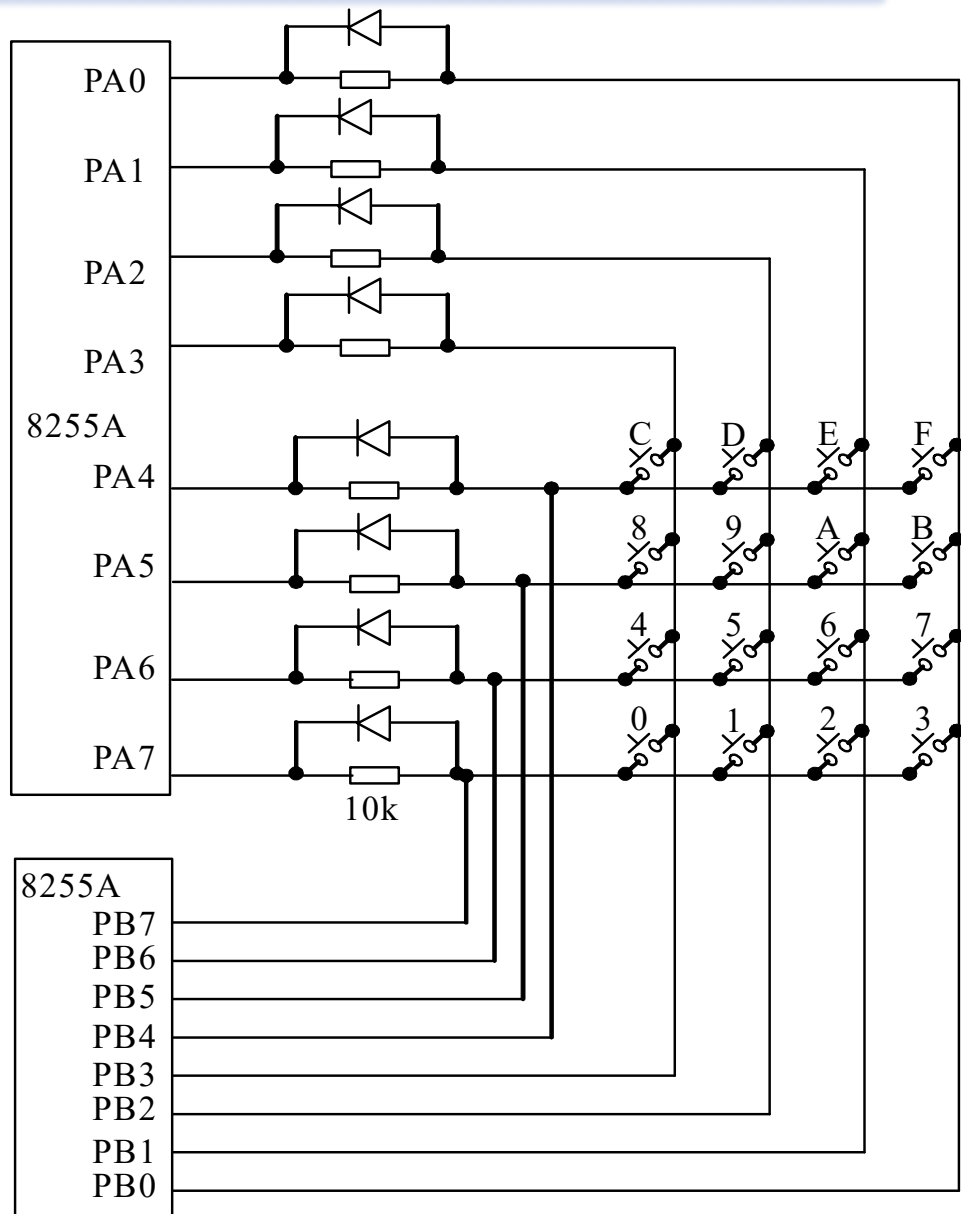
EXIT: HLT

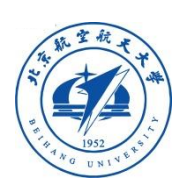
```
CODE ENDS
      END
```

6.3.4 键盘接口-方法2 (了解)

■ 键盘接口电路





键盘扫描

识别键盘上哪个键被按下的过程称为键盘扫描，包括以下步骤：

1. 检测是否所有键都松开了，若没有则反复检测
2. 当所有键都松开时，再检测是否有键压下，若没有则反复检测
3. 若有键按下，要消除键抖动，确认有键压下
4. 对压下的键进行编码，将该键的行列信号转换成16进制码，由此确定哪个键被压下。如果出现多键按下的情况，只有在其它键释放后，仅一个键闭合时才把此键当做本次压下的键。
5. 该键释放后，再回到2

;端口地址

PORT_A EQU 0FF9H ;8255A端口A地址

PORT_B EQU 0FFBH ;8255A端口B地址

PORT_CTL EQU 0FFFH ;8255A控制口地址

;数据段， 键盘扫描码表

DATA SEGMENT

TABLE DB 77H, 7BH, 7DH, 7EH ; 0, 1, 2, 3

DB 0B7H, 0BBH, 0BDH, 0BEH ; 4, 5, 6, 7

DB 0D7H, 0DBH, 0DDH, 0DEH ; 8, 9, A, B

DB 0E7H, 0EBH, 0EDH, 0EEH ; C, D, E, F

DATA ENDS

;堆栈段

STACK SEGMENT STACK

DW 50 DUP(0)

TOP_STAC LABEL WORD

STACK ENDS

;代码段

CODE SEGMENT

ASSUME CS:CODE, DS:DATA, SS:STACK

START: MOV AX, STACK

MOV SS, AX

MOV AX, DATA

MOV DS, AX

;初始化8255A, 方式0, A口作输出, 端口B作输入

MOV DX, PORT_CTL ;指向控制口

MOV AL, 10000010B ;控制字

OUT DX, AL ;写入控制字

; ;从行扫描开始

MOV AL, 0F0H

MOV DX, PORT_A

OUT DX, AL

;等待松开所有按键

MOV DX, PORT_B

WTRLS: IN AL, DX

CMP AL, 0F0H ;AL低4位为0

JNE WTRLS

;检查是否有按键按下

CHECKKEY: IN AL, DX

CMP AL, 0F0H

JE CHECKKEY

;等待一段时间后, 检查按键是否依然按下, 以消除抖动

MOV CX, 16EAH

DELAY: LOOP DELAY

IN AL, DX

CMP AL, 0F0H

JE CHECKKEY ;已经松开按键, 再次检查并等待

;有键按下，暂存行扫描结果

MOV BL, AL ;AL低4位为0

;扫描列

MOV AL, 0FH

MOV DX, PORT_A

OUT DX, AL

MOV DX, PORT_B

IN AL, DX ;高4位为0

;合并行扫描（BL高4位）和列扫描（AL低4位）结果

;得到按键的扫描码

OR AL, BL

;根据AL中的扫描码和TABLE，查出对应的数值

MOV BX, 0FH

NEXT_TRY: CMP AL, TABLE[BX]

JE DONE

DEC BX

JNS NEXT_TRY

MOV AH, 1

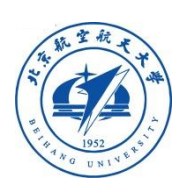
JMP EXIT

DONE: MOV AH, 0

EXIT: HLT

CODE ENDS

END START



总结

- 输入输出基本概念和功能
- 端口及IO空间，8086的IO端口地址范围
- 输入输出基本方式：程序方式（包括无条件方式和查询方式）、中断方式和DMA方式
- 8255A的引脚及内部结构
- 8255A的内部寄存器和工作方式
- 8255A应用举例